Assignment 3 – Computer Vision Classification through convolutional networks

Objectives

- To develop a complete system for the classification of aerial images acquired by RGB cameras.
- The goal is to build a residual convolutional network with some minimum specifications. In addition, two training strategies will have to be compared:
    1. The first strategy involves pre-training on the AID dataset and fine-tuning on the UC dataset.
    2. The second strategy involves training exclusively on the UC dataset.

Determine:

- The best model by evaluating both strategies on the validation set of the UC dataset, selecting the one with the highest F1-score.

Dataset

- The Aerial Image Dataset (AID) is divided into train and validation following the 70-30 ratio.
- While the UC Merced Land Use Dataset is divided into train, validation and test following the 70-15-15 ratio. These partitions are the same for both the first and second strategies. In addition, the images have been converted from tif to png.

Gabriele Lo Cascio

Introduction – Neural Networks

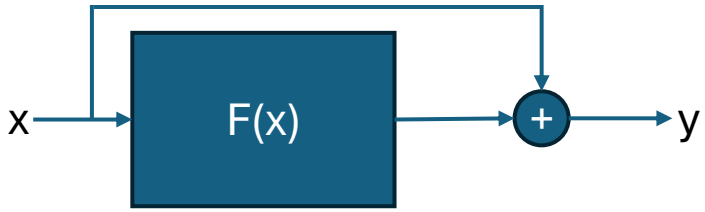
- The increasing availability of data has led to the use of data-driven paradigms. The fascination of neural networks derives from the fact that it is no longer strictly necessary to have a domain expert to guide the analysis of the data. This is because neural networks allow you to extract features and analyze patterns automatically.
- From the perceptron hyperplane to modern architecture, the goal remains to approximate a function that maps inputs to outputs. This function, extremely complex and highly non-linear, can be understood as a transformation that models the data space and generalizes, with a certain confidence, its properties.
- The capacity of the model depends on its complexity, for example on the number of parameters and the architecture.
- Optimization is done by fixing a loss and iteratively updating the parameters by backpropagation and gradient descent:

$$\overline{w}_{new} = \overline{w}_{old} - \eta \frac{\partial L}{\partial \overline{w}_{old}}$$ where η is called learning rate.

Deep networks with many nonlinear transformations tend to suffer from the problem of vanishing gradient: gradients can attenuate exponentially through the chain rule and become numerically negligible, preventing significant updates in the deep layers.

Introduction – Residual Networks

- The idea is to create skip connections: direct connections that connect a layer's input to its output by means of a sum.
- In this way, during the backpropagation process, that direct link provides an alternate path for the gradient to flow, mitigating the problem of vanishing gradient.
- This choice is not only convenient from an operational point of view, but is theoretically justified: the network learns, with respect to identity, a correction called residual.



Residual block diagram

- The network output is:

$$y = H(x) = F(x) + x$$

- So the network learns:

$$F(x) = H(x) - x$$

Analysis of AID and UC datasets

- The AID dataset collects RGB images with dimensions 600x600x3, while the UC dataset collects 256x256x3. It was decided not to use any pre-processing technique (unless scaling) and to reduce the spatial dimension to 224x224x3.
- This choice stems from wanting to reduce the number of parameters of the convolutional backbone to limit the computational load.
- In addition, several historical papers, including [1] and [2], employ the same spatial resolution. Therefore, it will be possible to compare the developed method with other works in a coherent way.
- Following [1], a data augmentation technique was employed, albeit less sophisticated. In particular, you flip images horizontally and apply rotations of a few degrees. In the AID dataset, there is a corrupt image that has been removed (railwaystation_7).
- In addition, a certain imbalance between the classes of the AID dataset was noted. This was handled by calculating a weight to be associated with each class which was then used for the calculation of the loss:

$$w_i = \frac{N_{img_T}}{N_c \cdot N_{img_{c_i}}}, \qquad Loss = -\sum_{i=1}^{N_c} w_i \cdot y_i \cdot \log(\hat{y}_i)$$

The idea is that a class with many champions should have less weight. The same procedure was also used for the UC dataset despite being balanced, this is to avoid complicating the code.

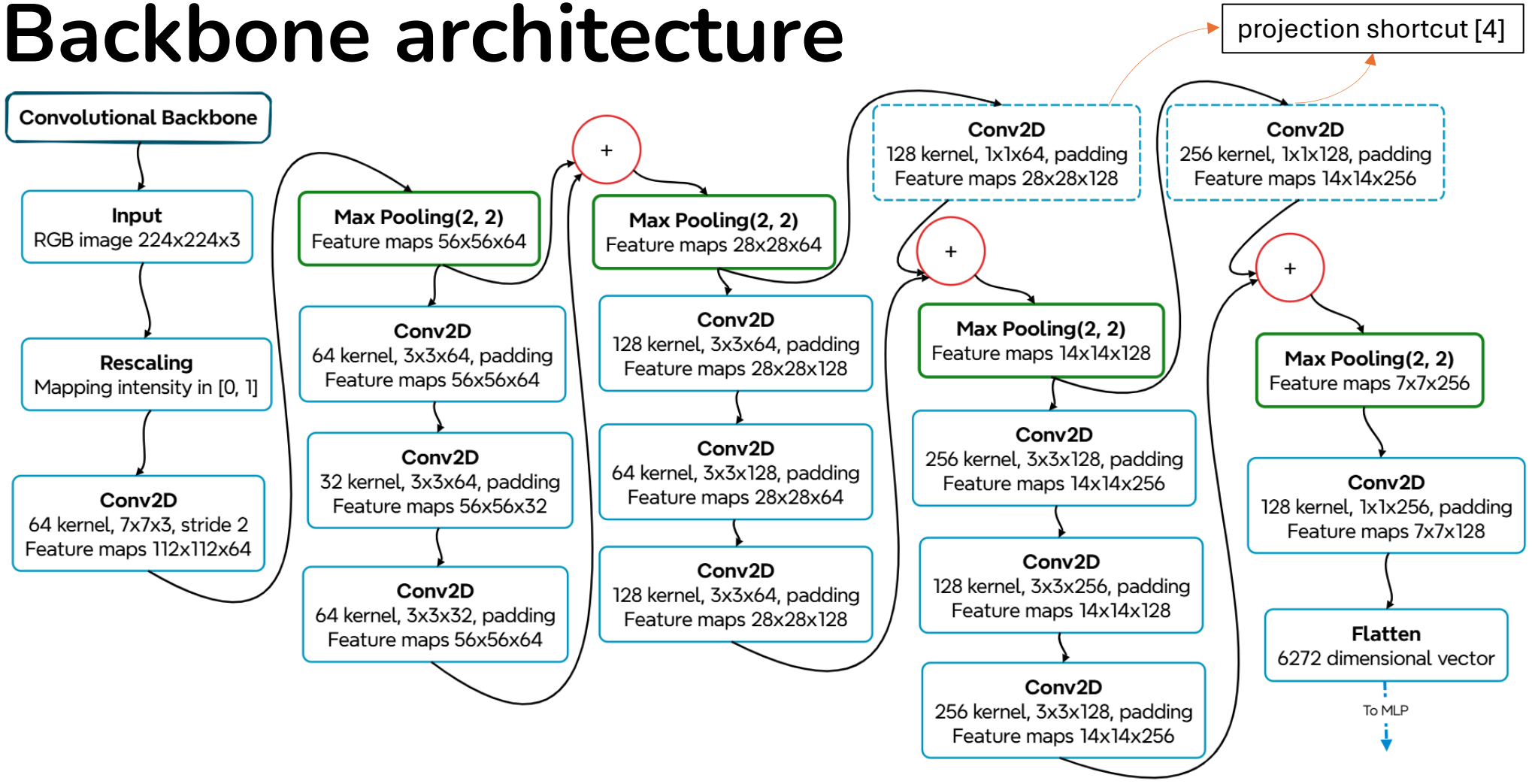
Models and implementation choices

- The architecture of the proposed model consists of a convolutional residual network, for feature extraction, followed by a multi-layer perceptron (MLP) to perform classification.
- Below are the two strategies in detail:
  - First strategy:

    For pre-training, the convolutional backbone is considered followed by an MLP with 30 neurons in the last layer, this is due to the classes of the AID dataset.
    For fine-tuning, the convolutional block is frozen and the last layer of the MLP is replaced with one with 21 neurons, this because of the classes of the UC dataset. Then only the MLP classifier layers will be trained.
  - Second strategy:

    We consider the convolutional backbone followed by an MLP with 21 neurons in the last layer, this is due to the classes of the UC dataset. Then you train both the backbone and the dense layers.
- Note that:
  - The convolutional backbone is structurally the same for both strategies.
  - Considering [5], 16 images were chosen as the batch size.
  - Early stopping, progressive learning rate reduction and saving metrics for each era have been implemented.

Backbone architecture

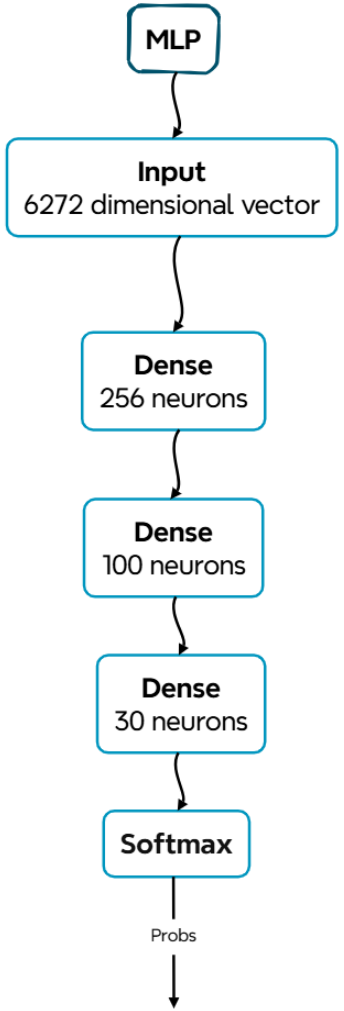


Batch normalization and ReLU were employed after each convolution.

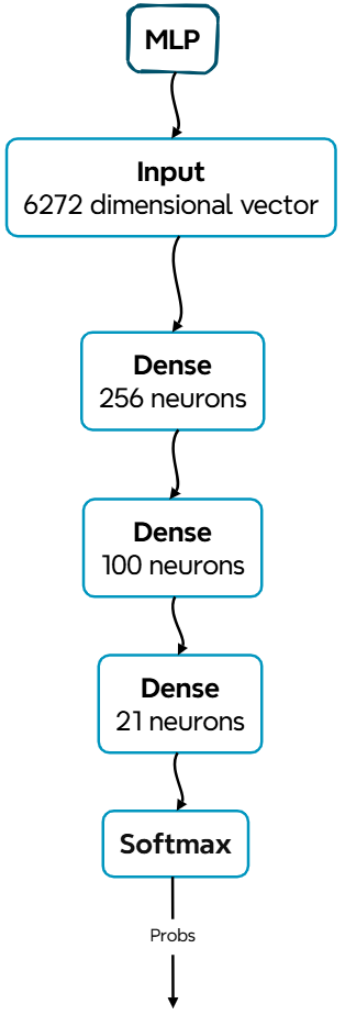
Initially, scaling is applied as reported in [3]. Next, the first convolutional block has a dual purpose: to replace the classic explicit pre-processing techniques by applying learned filters, but also to reduce the spatial resolution. In this way, the rest of the network focuses on the macroscopic details, while here the finer details are analyzed. The structure takes its cue from the graphs shown in [4], as well as doubling the kernels after halving the spatial resolution of the maps. The residual blocks follow an hourglass structure to reduce the number of parameters.

MLP architecture

Pre-training classifier

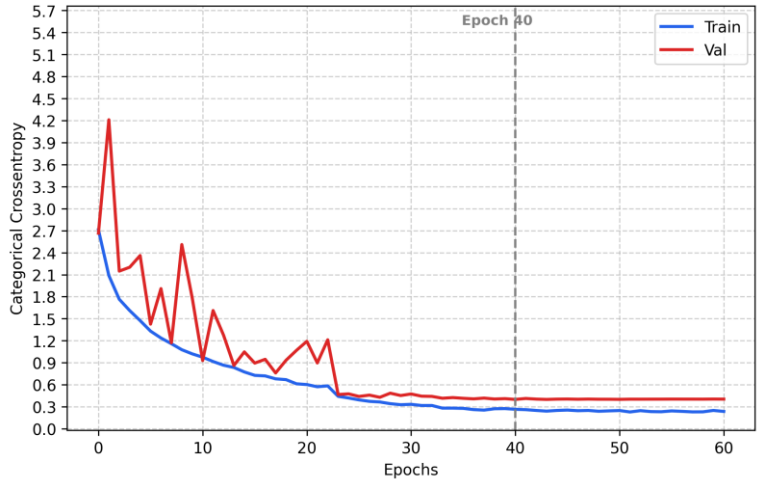


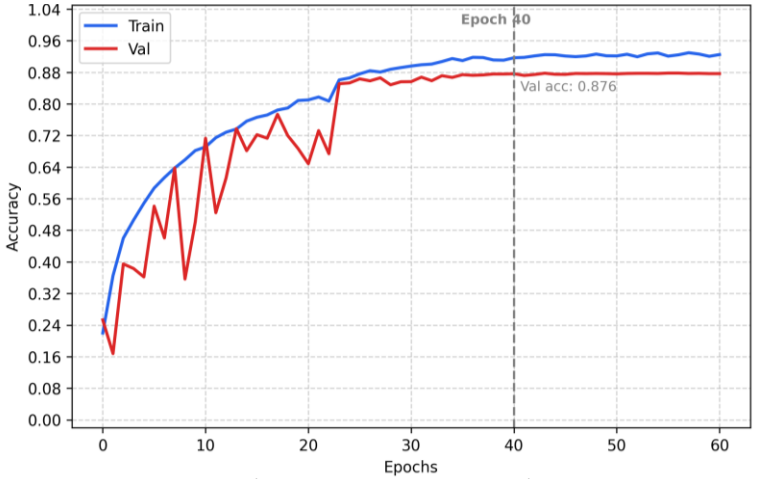
Final Classifier



- The MLPs strongly conditioned the construction of the backbone: it was avoided to have many parameters in the first dense layer because of flattening.
- In addition, thanks to this stratagem, there is a similar total number of parameters between the backbone and MLP. As a result, the model will have a capacity evenly distributed between the feature extractor and the classifier.
- Batch normalization, ReLU, and Dropout were employed after each layer, except for the last one.

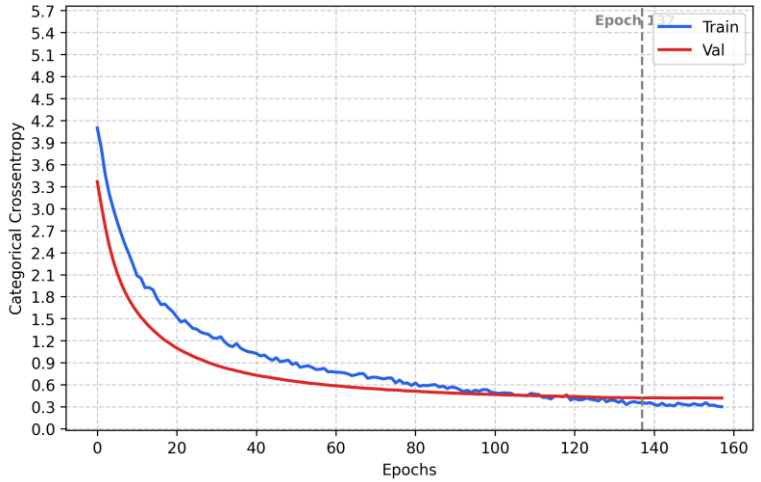
Andamento loss e accuracy

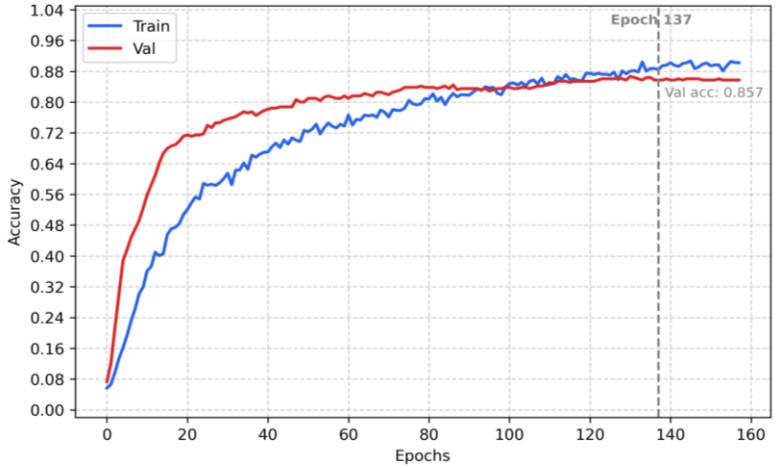




Pre-trained network: GaidNet

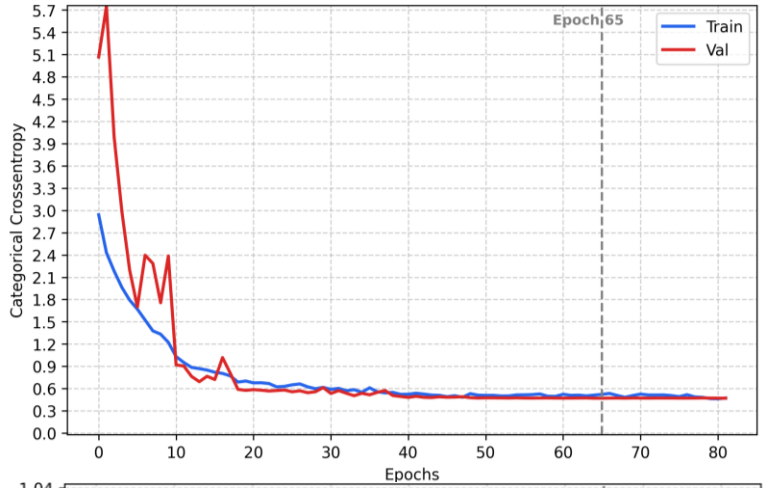
The trends on the train set are quite typical. While those on the validation set are particularly noisy, but stabilize after a certain era. Early stopping stopped training when improvement became negligible.

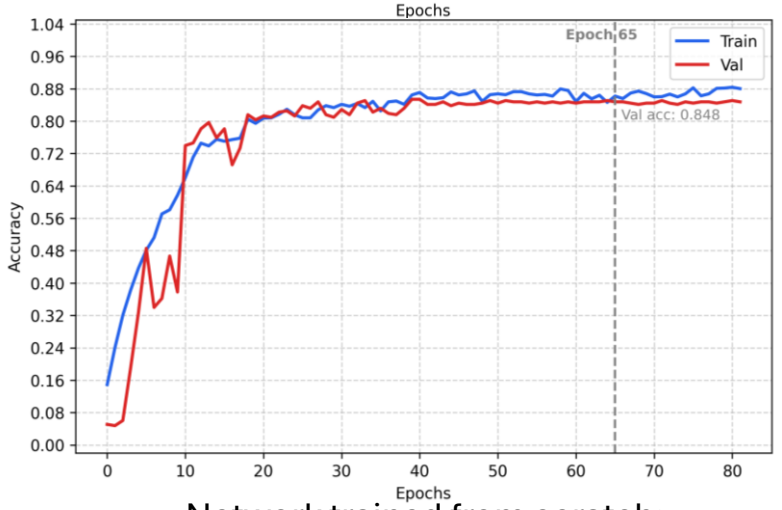




Fine-tuned net: GumerNet

The dropout of the MLP during training does not allow all neurons to participate. While for the validation set, in the inference phase, the full power of the network is exploited. So higher capacity, combined with pre-training, means that the validation loss is initially less than the train loss. This denotes the advantage and goodness of pre-training. Here too, early stopping has intervened effectively.





Network trained from scratch: ZeroGumerNet

Targeted training on the specific dataset leads faster to convergence: at epoch 20 the performance that the fine-tuned model achieves at epoch 40 is achieved.

Model selection

- Evaluating the two strategies on the validation set of the UC dataset, the following results are obtained:
- The fine-tuned network shows a slight superiority over the network trained from scratch on the UC dataset. The first strategy is the winning one.
- Further tests should be carried out, for example by comparing other finetuning strategies.

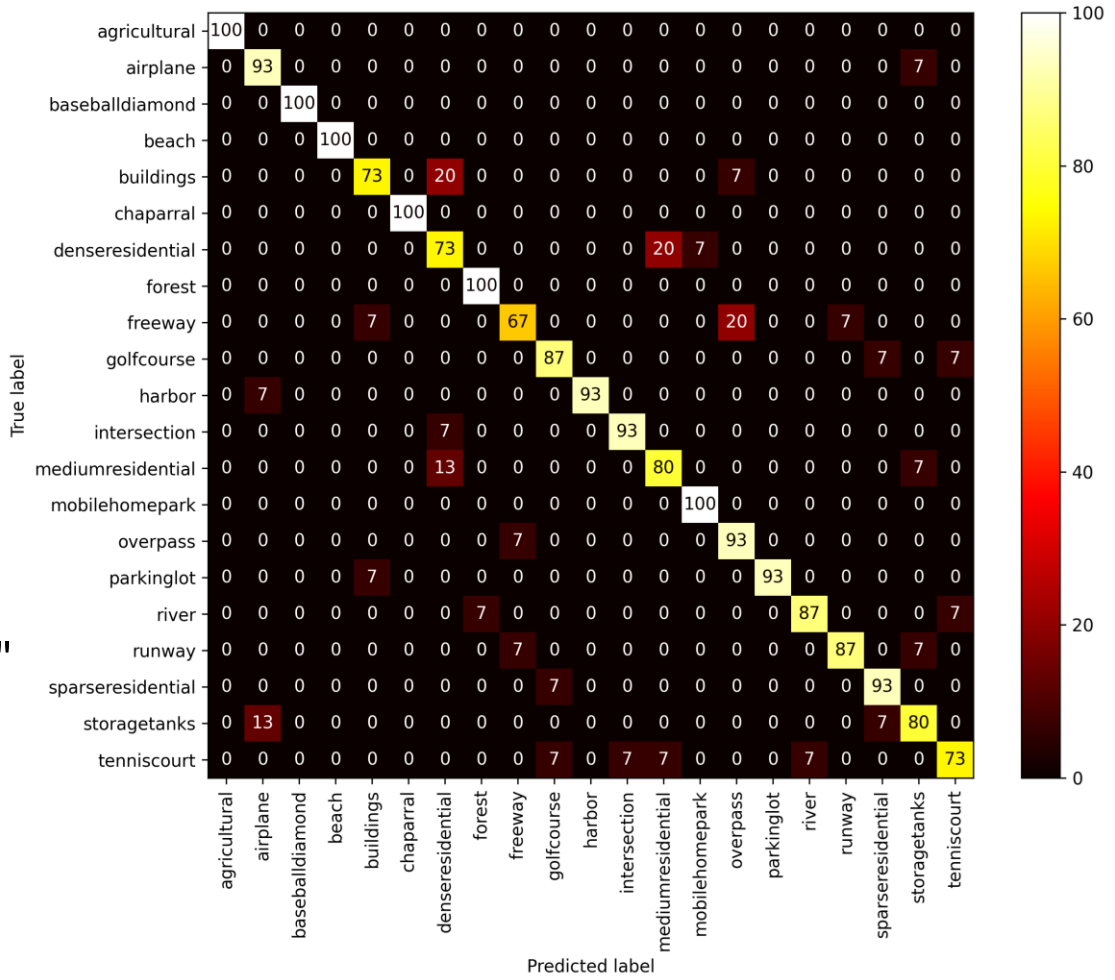
- An interesting test could be to increase the number of parameters in the model and check to what extent the performance differs. An interesting comparison would also be regarding the number of eras and the training times used.

model	F1 macro	Precision	Recall	Accuracy
GumerNet	0.852	0.860	0.857	0.857
ZeroGumerNet	0.839	0.853	0.848	0.848

Results on the test set

model	F1 macro	Precision	Recall	Accuracy
GumerNet	0.889	0.892	0.889	0.889

- All classes have a correctness rate ≥73%.
- Only the "freeway" class is the only one with a lower rate, i.e. 67%.
- It should be noted that although the class is incorrect, the macro category is correct: "buildings" with "denseresidential" or "freeway" with "overpass" or "denseresidential" with "mediumresidential".



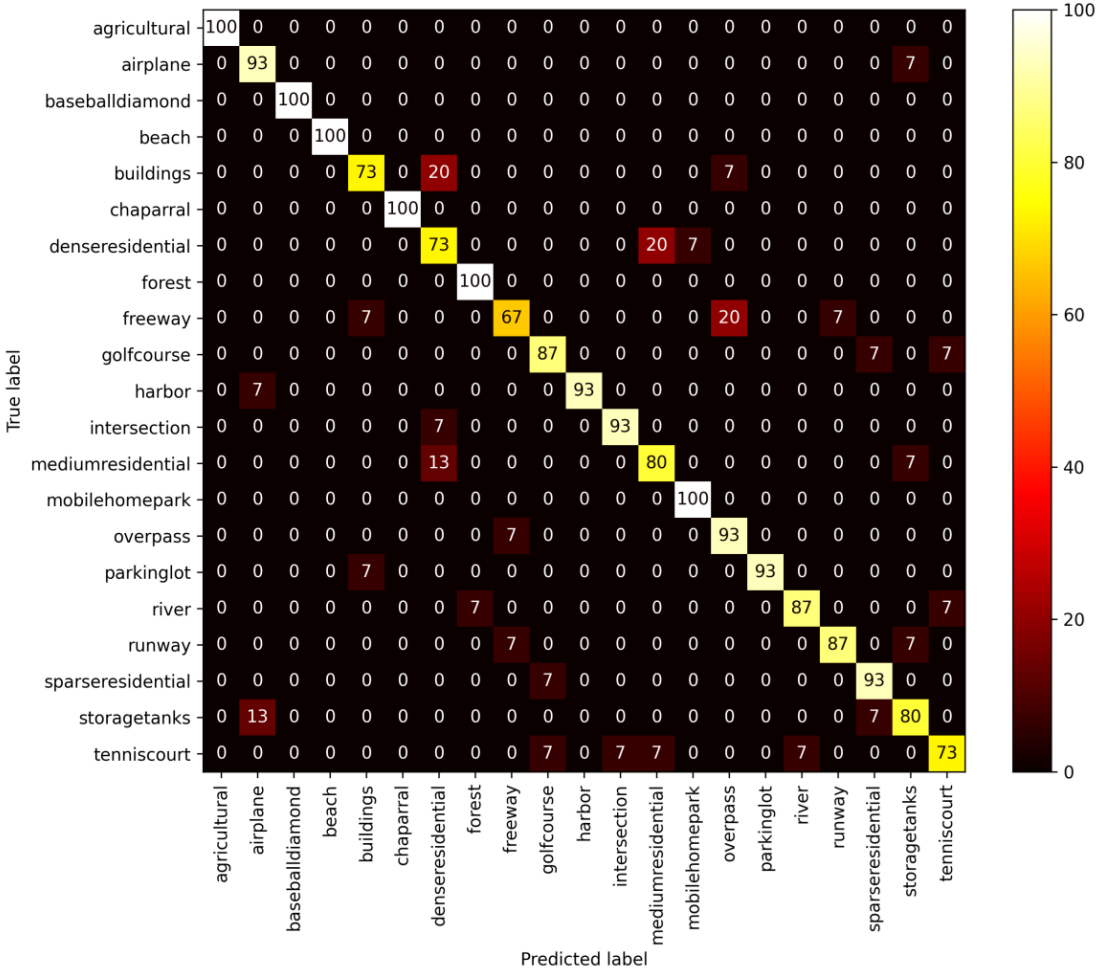
GumerNet confusion matrix.

Test out of bounds

Screenshot Punta Raisi

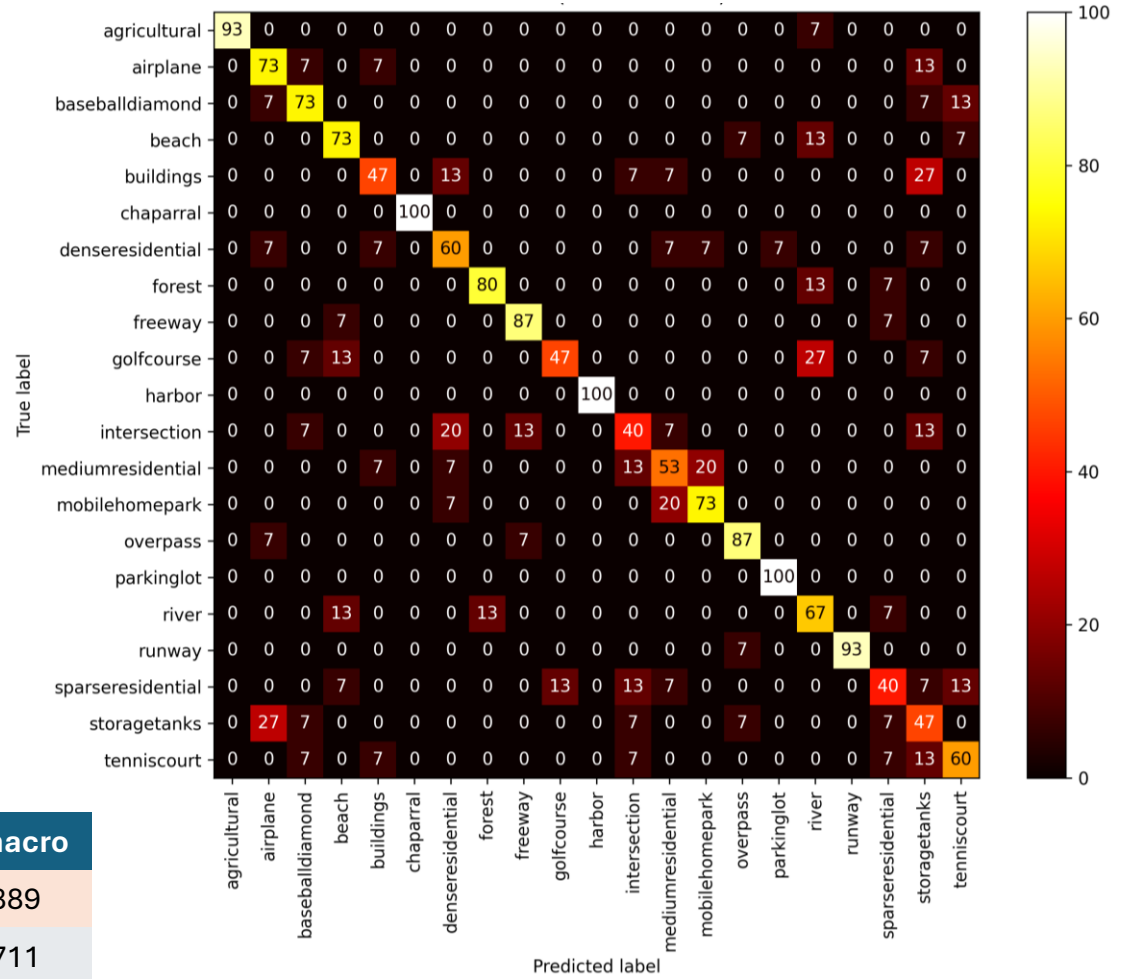
```
<- Classifico punta_raisi.png ->
appartiene alla classe: airplane ->
```

Comparison of the previous assignment model



GumerNet confusion matrix.

model	F1 macro
GumerNet	0.889
SoftVoting	0.711

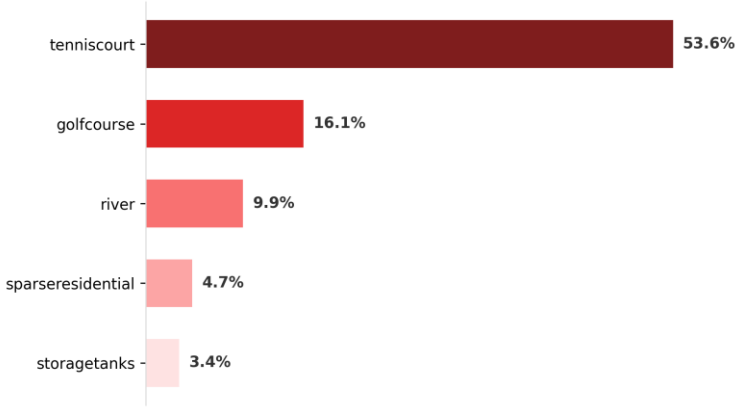


Confusion matrix related to the model of the previous assignment.

- The previous model was trained and tested on the same partition of the data.
- The increase in performance is about 18%. The impact on the confusion matrix is obvious. One reason is the small size of the train set partition that has not been given augmentation. This involves random rotations, and SIFT descriptors are invariant with respect to these transformations.

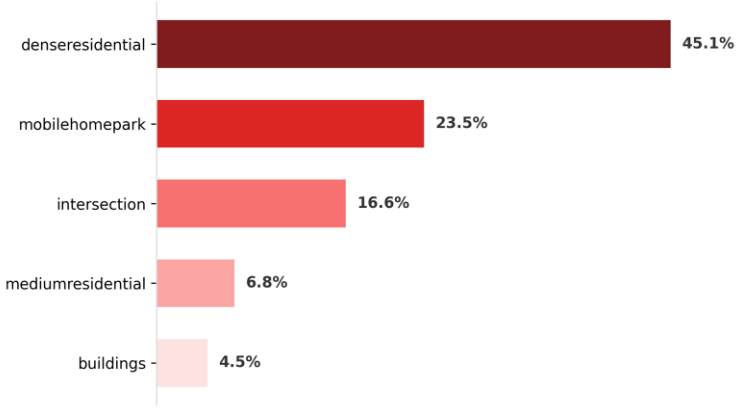
GumerNet – qualitative results from the test set

True label: golfcourse.
Predicted label: tenniscourt.



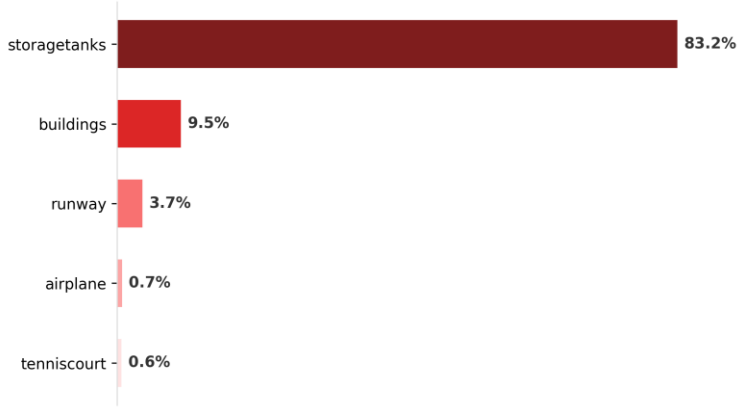
The true class is the second one foretold.

True label: denseresidential.
Predicted label: denseresidential.



The class chosen is correct, and the top ones are consistent with the image.

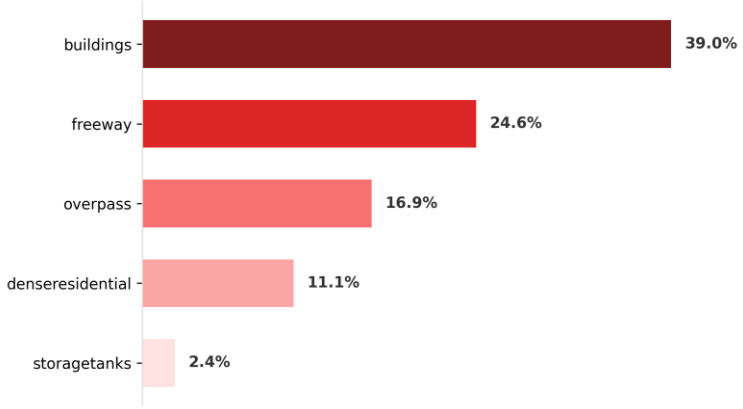
True label: storagetanks.
Predicted label: storagetanks.



The class you choose is extremely more likely than the others.

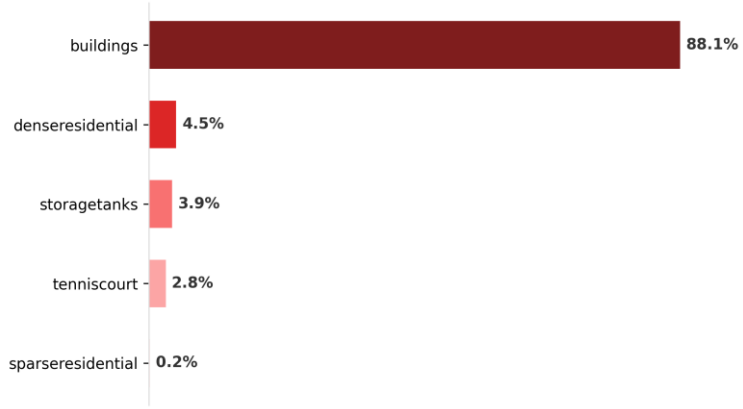
GumerNet – qualitative results from the test set

True label: freeway.
Predicted label: buildings.



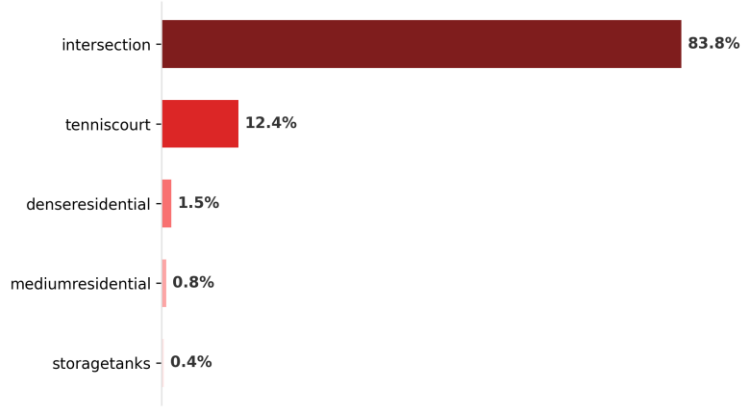
The true class is the second one foretold. It might look like a roof from above when you compare it to the image on the right.

True label: buildings.
Predicted label: buildings.



The class you choose is extremely more likely than the others.

True label: tenniscourt.
Predicted label: intersection.



The true class is the second one foretold. In addition, the predicted class is consistent with the image.

Conclusions and future ideas

- The comparison with the model of the previous assignment showed that the approach used with neural networks produced good results: this can be seen from the significant jump in performance of +18% (what happened with AlexNet was observed in miniature).
- Although the first strategy is the winning one, the second does not have significantly different performances. On the other hand, the fine-tuned model was subjected to a much larger amount of data than the one trained from scratch.
- Therefore, although the two datasets are related to aerial images, it is also necessary to intervene on the backbone, so as to extract peculiar features of the UC dataset: using a finetuning of the backbone could also be advantageous.
- Furthermore, even before trying other training and fine-tuning strategies, the two models could be tested on other datasets and thus evaluate their goodness.
- At a later stage, one could think of employing more robust data augmentation techniques, as in [1]. Alternatively, the number of residual blocks and in general the number of model parameters could be increased, combining regularization techniques on weights (which were not used in this assignment).

Bibliography

[1] NIPS-2012-imagenet-classification-with-deep-convolutional-neural-networks-Paper

[2] Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG)

[3] TensorFlow Tutorial

[4] Deep Residual Learning for Image Recognition (ResNet)

[5] Batch dimension reference

Disclaimer

Generative models such as Gemini and Claude have been used to:

- Implementation of the code parts involving the matplotlib library and splitfolders
- Quickly interact with library documentation
- Troubleshoot issues with the TensorFlow library
- Managing TensorFlow Library Functions and Objects

The Perplexity search engine was used to find and analyze scientific papers.